

Python SQL Information Extraction Assignment

Intern: Sumit Dhar

Date: 26th September, 2025

Objective:

The objective of this project is to build a lightweight Python-based tool that can automatically extract structured metadata from SQL files. By parsing SQL scripts, the tool identifies key database objects (tables, columns, procedures, views) and DML statements (INSERT, UPDATE, DELETE), and exports the information into a structured JSON format.

Creating a strong MVP:

- **CREATE** → Table + Procedure + View.
 - **INSERT** → Single-row insert.
 - **UPDATE** → Salary modification.
 - **DELETE** → Row deletion.
 - **Table:** Has columns with datatypes (easy to parse).
 - **Stored Procedure:** Has parameters + body (tests regex multiline).
 - **View:** Has a simple query definition.
 - Covers both **DDL (Data Definition Language)** and **DML (Data Manipulation Language)**.
-

Extracting Tables

```
CREATE\s+TABLE\s+(\w+)\s*\(((.*?)\)\);
```

- **Group 1** → Table name (**Employees**)
- **Group 2** → Column definitions (**EmpID INT PRIMARY KEY, ...**)
- We'll split Group 2 by commas → get column name + type.

```
-- Create a table
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(100),
    DeptID INT,
    Salary DECIMAL(10,2)
);
```

```
def extract_tables(sql):
    tables = []
    pattern = re.compile(r"CREATE\s+TABLE\s+(\w+)\s*\(((.*?)\)\);", re.IGNORECASE | re.DOTALL)
    matches = pattern.findall(sql)
    for name, cols in matches:
```

```

columns = []
for col in re.split(r",\s*(?![^\)]*\s)", cols):
    col = col.strip()
    if col:
        parts = col.split(None, 1) # split into name + type
        if len(parts) == 2:
            columns.append({"name": parts[0], "type": parts[1]})
tables.append({"name": name, "columns": columns})
return tables

```

Extract Stored Procedures

```
CREATE\s+PROCEDURE\s+([A-Za-z0-9_]+\s*(.*)\s*AS\s*BEGIN(.*)END;
```

- **Group 1** → Procedure name (**GetEmployeeDetails**)
- **Group 2** → Parameters (**@EmpID INT, @DeptID INT**)
- **Group 3** → Body (**SELECT EmpID, ...**)

```

-- Create a stored procedure
CREATE PROCEDURE GetEmployeeDetails
    @EmpID INT,
    @DeptID INT
AS
BEGIN
    SELECT EmpID, EmpName, Salary
    FROM Employees
    WHERE EmpID = @EmpID AND DeptID = @DeptID;
END;

```

```

def extract_procedures(sql):
    # Extract procedures (with full definition)
    procedure_pattern = re.compile(
        r"CREATE\s+PROCEDURE\s+([A-Za-z0-9_]+\s*(.*)\s*AS\s*BEGIN(.*)END;",
        re.IGNORECASE | re.DOTALL
    )
    procedures = []
    for match in procedure_pattern.finditer(sql):
        name = match.group(1).strip()
        params = match.group(2).strip()
        body = match.group(3).strip()
        query = f"CREATE PROCEDURE {name} {params} AS BEGIN {body} END;"
        definition = " ".join(query.split())
        parameters = [p.strip() for p in params.replace("\n", " ").split(",") if p.strip()]
        procedures.append({
            "name": name,
            "params": parameters,
            "definition": definition
        })
    return procedures

```

Extract Views

CREATE\s+VIEW\s+(\w+)\s+AS\s+(. *?) ;

- **Group 1** → View name (**HighSalaryEmployees**)
- **Group 2** → Definition (**SELECT EmpID, EmpName ...**)

```
-- Create a view
CREATE VIEW HighSalaryEmployees AS
SELECT EmpID, EmpName, Salary
FROM Employees
WHERE Salary > 50000;
```

```
def extract_views(sql):
    views = []
    pattern = re.compile(r"CREATE\s+VIEW\s+(\w+)\s+AS\s+(.*?)";, re.IGNORECASE | re.DOTALL)
    matches = pattern.findall(sql)
    for name, definition in matches:
        views.append({"name": name, "definition": definition.strip()+";"})
    return views
```

Extract INSERT

```
INSERT\s+INTO\s+(\w+)\s*\((\.*?)\)\s*\s*VALUES\s*\((\.*?)\);
```

- **Group 1** → Table name (**Employees**)
- **Group 2** → Column list (**EmpID, EmpName, DeptID, Salary**)
- **Group 3** → Values (**1, 'Emp1', 101, 60000**)

For MVP $\rightarrow$ we can just store "**query**": full match.

```
-- Insert some data
INSERT INTO Employees (EmpID, EmpName, DeptID, Salary)
VALUES (1, 'Emp 1', 101, 60000);

INSERT INTO Employees (EmpID, EmpName, DeptID, Salary)
VALUES (2, 'Emp 2', 102, 45000);
```

```
def extract_inserts(sql):
    inserts = []
    pattern = re.compile(r"(INSERT\s+INTO\s+\w+\s*\s*(.*?\s)\s*VALUES\s*(.*?\s);)",
re.IGNORECASE | re.DOTALL)
    matches = pattern.findall(sql)
    for query in matches:
        inserts.append({"type": "INSERT", "query": query.strip()})
    return inserts
```

Extract UPDATE

`UPDATE\s+(\w+)\s+SET\s+(.*?)\s+WHERE\s+(.*?) ;`

- **Group 1** → Table (**Employees**)
- **Group 2** → Set clause (**Salary = 65000**)
- **Group 3** → Where clause (**EmpID = 3**)

```
-- Update data
UPDATE Employees
SET Salary = 65000
WHERE EmpID = 3;
```

```
def extract_updates(sql):
    updates = []
    pattern = re.compile(r"(UPDATE\s+\w+\s+SET\s+.*?\s+WHERE\s+.*?);", re.IGNORECASE | re.DOTALL)
    matches = pattern.findall(sql)
    for query in matches:
        updates.append({"type": "UPDATE", "query": query.strip()})
    return updates
```

Extract DELETE

`DELETE\s+FROM\s+(\w+)\s+WHERE\s+(.*?) ;`

- **Group 1** → Table (**Employees**)
- **Group 2** → Where condition (**EmpID = 4**)

```
-- Delete data
DELETE FROM Employees
WHERE EmpID = 4;
```

```
def extract_deletes(sql):
    deletes = []
    pattern = re.compile(r"(DELETE\s+FROM\s+\w+\s+WHERE\s+.*?);", re.IGNORECASE | re.DOTALL)
    matches = pattern.findall(sql)
    for query in matches:
        deletes.append({"type": "DELETE", "query": query.strip()})
    return deletes
```

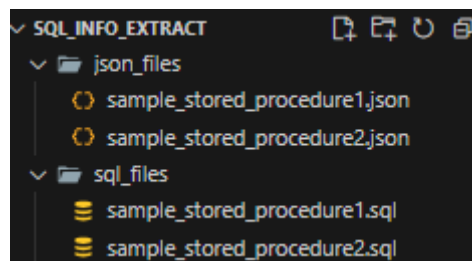
Step 1: Created the folder structure:

```
Sql_info_extract/  
├── sql_files/  
│   ├── sample_stored_procedure1.sql  
│   ├── sample_stored_procedure2.sql  
│   ├── .  
│   └── .  
├── json_files/  
│   ├── sample_stored_procedure1.json  
│   ├── sample_stored_procedure2.json  
│   ├── .  
│   └── .  
└── extractor.py
```

Step 2: Run **extractor.py** with by specifying the path of **'sql'** file.

python extractor.py

```
(testing_DG_1) PS F:\Internship\Python\sql_info_extract> python .\extractor.py  
Processing: sql_files/sample_stored_procedure1.sql → Saved to: json_files\sample_stored_procedure1.json  
Processing: sql_files/sample_stored_procedure2.sql → Saved to: json_files\sample_stored_procedure2.json  
  
Extraction complete! JSON saved to json_files  
(testing_DG_1) PS F:\Internship\Python\sql_info_extract> █
```



Output:

```
{
  "tables": [
    {
      "name": "Employees",
      "columns": [
        {
          "name": "EmpID",
          "type": "INT PRIMARY KEY"
        },
        {
          "name": "EmpName",
          "type": "VARCHAR(100)"
        },
        {
          "name": "DeptID",
          "type": "INT"
        },
        {
          "name": "Salary",
          "type": "DECIMAL(10,2)"
        }
      ]
    }
  ],
  "procedures": [
    {
      "name": "GetEmployeeDetails",
      "params": [
        "@EmpID INT",
        "@DeptID INT"
      ],
      "definition": "CREATE PROCEDURE GetEmployeeDetails @EmpID INT, @DeptID INT AS BEGIN SELECT EmpID, EmpName, Salary FROM Employees WHERE EmpID = @EmpID AND DeptID = @DeptID; END;"
    }
  ]
},
```

```
  "views": [
    {
      "name": "HighSalaryEmployees",
      "definition": "SELECT EmpID, EmpName, Salary\nFROM Employees\nWHERE Salary > 50000;"
    }
  ],
  "dml": [
    {
      "type": "INSERT",
      "query": "INSERT INTO Employees (EmpID, EmpName, DeptID, Salary)\nVALUES (1, 'Emp 1', 101, 60000);"
    },
    {
      "type": "INSERT",
      "query": "INSERT INTO Employees (EmpID, EmpName, DeptID, Salary)\nVALUES (2, 'Emp 2', 102, 45000);"
    },
    {
      "type": "INSERT",
      "query": "INSERT INTO Employees (EmpID, EmpName, DeptID, Salary)\nVALUES (3, 'Emp 3', 103, 47000);"
    },
    {
      "type": "INSERT",
      "query": "INSERT INTO Employees (EmpID, EmpName, DeptID, Salary)\nVALUES (4, 'Emp 4', 104, 50000);"
    },
    {
      "type": "INSERT",
      "query": "INSERT INTO Employees (EmpID, EmpName, DeptID, Salary)\nVALUES (5, 'Emp 5', 105, 74000);"
    },
    {
      "type": "UPDATE",
      "query": "UPDATE Employees\nSET Salary = 65000\nWHERE EmpID = 3;"
    },
    {
      "type": "DELETE",
      "query": "DELETE FROM Employees\nWHERE EmpID = 4;"
    }
  ]
}
```

Summary:

This project provides a simple yet effective way to analyze SQL files and generate machine-readable metadata. Using regular expressions, the script scans SQL definitions and extracts important details such as:

- **Tables** (names, columns, and datatypes)
- **Stored Procedures** (name, parameters, body definition)
- **Views** (name and definition)
- **DML statements** (INSERT, UPDATE, DELETE with queries)

The extracted data is then saved in JSON format, enabling easier analysis, documentation, or integration with other tools. The solution is kept minimal and dependency-free, making it portable and easy to extend for more advanced SQL parsing in the future.
